RW-08 â€” Search-latency / `/search/sync` reset over tunnel â€” READ-ONLY diagnosis

Revision: 1 **Last modified:** 2026-06-16T10:00:00Z **Status:** diagnosis only â€” NO code edits (another agent owns `download-proxy/`) **Author:** background diagnostic subagent. Anti-bluff (Â§11.4.6): every claim cites a real file:line, the running tunnel log, or the source. Hypotheses are labelled as hypotheses; the confirmed facts are stated as facts. **Scope:** characterize RW-08 (`docs/REMAINING_WORK_PLAN.md` Â§RW-08) and propose a fix direction for the main agent to implement later.

---

1. The symptom (from the plan + live evidence)

- Broad `ubuntu` search â‰ˆ **67 s** in-container wall-clock (plan RW-08).
- Over the SSH tunnel, `POST /search/sync` **resets** (`ConnectionResetError`) at ~13â€“40 s; browser console shows `ERR_INCOMPLETE_CHUNKED_ENCODING`.
- The SSE path (`POST /search` + `GET /search/stream/{id}`) **survives** and the dashboard uses it; scripted/`curl` callers of `/search/sync` hang/reset.

---

2. Root cause â€” there are TWO distinct, independent problems

2a. The reset is an SSH-tunnel idle-timeout artifact, NOT a server fault (FACT)

`/search/sync` (`download-proxy/src/api/routes.py:377-406`) is **fully blocking**: it does `metadata = await orch.search(...)` (line 400), and `SearchOrchestrator.search` (`search.py:903-929`) calls `start_search` + `await self._run_search(...)` and only returns after the **entire** 29-tracker fan-out + dedup completes. FastAPI buffers the whole `SearchResponse` and sends the first bytes **only at the end**. So the HTTP response sends **zero bytes for the full ~67 s**.

An SSH `-L` tunnel (and any intermediate proxy/NAT) drops a connection that is idle â€” no bytes in either direction â€” past its keepalive window. With nothing flowing for tens of seconds the tunnel tears the socket down; the client sees `ConnectionReset` / `ERR_INCOMPLETE_CHUNKED_ENCODING` while the **server side still returns 200** once it finishes (it just has no socket left to write to). This is corroborated by the live keepalive supervisor log `qa-results/tunnel-keepalive-testsession.log`, which shows the tunnel itself repeatedly going â€œhealth check FAILED on 127.0.0.1:7187 â€” re-establishingâ€¦ and recovering â€” i.e.Â the transport layer is intermittently dropping, independent of any single request. **Conclusion: the reset is transport-layer (idle tunnel), not an application bug. The previously-established â€œconsole `ERR_INCOMPLETE_CHUNKED_ENCODING` is an SSH-tunnel artifact, server-side 200â€¦ finding holds.**

Why the SSE path survives: `GET /search/stream/{id}` streams frames as trackers complete AND emits `: keepalive` comment lines (the same keepalive idiom is visible at

`routes.py:99` for the theme SSE stream). Continuous bytes keep the tunnelâ€™s idle timer from firing â€” so the SSE socket never goes quiet long enough to be reset. This is the structural reason the dashboard (SSE) works while `/search/sync` (one big blocking response) resets.

2b. The ~67 s wall-clock is the 29-provider fan-out cost (FACT + arithmetic)

Default fan-out is **29 providers**: 24 live public trackers (`PUBLIC_TRACKERS` = 38, minus `DEAD_PUBLIC_TRACKERS` = 14, default-excluded) + up to 4 private + Jackett (`_get_enabled_trackers`, `search.py:977-998`). They run under a **concurrency cap of 5** (`MAX_CONCURRENT_TRACKERS`, `search.py:611`) with a **per-public-tracker deadline of 60 s** (`PUBLIC_TRACKER_DEADLINE_SECONDS`, `search.py:1027,1082`, clamped 5â€“120).

So worst-case wall-clock â‰ˆ ceil(29 / 5) â‰ˆ **6 waves**, each bounded by the slowest tracker in the wave (â‰¤ 60 s + 10 s outer `asyncio.wait_for` margin at `search.py:1031-1034`). A single slow/borderline-alive tracker in each wave dominates that wave. The observed ~67 s is consistent with a handful of slow-but-not-dead public trackers each chewing toward the deadline across several waves. The dead-tracker exclusion already removed the 14 worst offenders; the remaining cost is the long tail of live-but-slow trackers running 6 waves deep at a 60 s ceiling.

---

3. What is NOT the cause (ruled out)

- **Not** an SSRF/auth/parse bug â€” the fan-out completes and returns 200 server-side.
- **Not** the rutracker ReDoS (RW-06) â€” that fix is deployed (the installed engine `config/qBittorrent/nova3/engines/rutracker.py` contains the `{0,512}` bound; verified this session) and its parse is sub-second; it is not the latency driver.
- **Not** unbounded fan-out â€” the semaphore (5) and the per-search cap (`MAX_CONCURRENT_SEARCHES`, `search.py:623`) are in place; the issue is total wall-clock under a 60 s per-tracker ceiling Ã— 6 waves, plus the blocking-response transport interaction.

---

4. Proposed fix direction (for the main agent â€” NOT implemented here)

Two independent levers; do BOTH for the cleanest result.

Fix A â€” keep the `/search/sync` socket warm (addresses the reset; highest value)

Make `/search/sync` stop being a single silent blocking response. Lowest-risk option: emit a periodic heartbeat so the tunnel never sees an idle socket. Either: - (preferred) keep `/search/sync` returning the final `SearchResponse` JSON, but stream it as a `StreamingResponse` that flushes a whitespace/newline (`" "/"\n"`) heartbeat every ~10 s while the fan-out runs, then the JSON body at the end â€” mirrors the `: keepalive` idiom already used for the theme SSE at `routes.py:99`; OR - **steer non-browser callers to the streaming endpoint** â€” document `POST /search` + `GET /search/stream/{id}` as the canonical path for slow/tunnelled clients, and treat `/search/sync` as a localhost/test convenience only. Note: the operatorâ€™s own `tunnel-keepalive` supervisor only keeps the *tunnel process* alive; it does not keep an *individual idle request* from being reset â€” so a per-request heartbeat (or SSE) is still required.

Fix B â€” cut the wall-clock (addresses the 67 s)

- Lower the default `PUBLIC_TRACKER_DEADLINE_SECONDS` (e.g.Â 60 â†’ 20â€“25) so a slow tracker canâ€™t dominate a wave; the dashboard already shows a `deadline_hit` chip (`search.py:1187`) so truncation stays honest/visible.
- Optionally raise `MAX_CONCURRENT_TRACKERS` (5 â†’ 8â€“10) to cut the wave count from ~6 to ~3â€“4 â€” bounded by Â§12.6 memory budget (each public tracker is a subprocess) and the existing event-loop-starvation guardrails noted at `search.py:617-623`.
- Capture before/after timing of a broad search (e.g.Â `ubuntu`) per RW-08 acceptance.

Fix C â€” regression guard (Â§11.4.85)

Add a chaos test for sync-over-slow-link: simulate a server that produces no bytes for > tunnel-idle-window and assert the heartbeat path keeps the socket alive (and/or assert the SSE path is unaffected). Add a deterministic latency test asserting the wave-count / deadline math so a future deadline-default regression is caught.

---

5. Root-cause hypothesis (one line)

The reset is an **SSH-tunnel idle-timeout** on `/search/sync`â€™s fully-blocking, zero-bytes-until-done response (server returns 200; the tunnel drops the idle socket mid-flight); the ~67 s wall-clock is the **29-provider fan-out** at 5-concurrent Ã— 60 s-per-tracker (~6 waves). Fix = heartbeat/stream the sync path (or steer callers to SSE) **and** tune the per-tracker deadline / concurrency. The SSE path is structurally robust because its continuous frames + `: keepalive` keep the tunnel warm.